

ForSyDe Shallow Reference Manual

ForSyDe DevTools Group

Klara Modin - Michel Delli Abo - Muhammad Afif Ramadhan

Samuel Miksits - Sebastian Thaeron - Zicong Zhang

October 2025

Contents

1	Syntax	2
1.1	Lexical Conventions	2
1.2	Concrete Syntax	3
2	Semantics	4
2.1	Abstract Syntax	4
2.2	Programmer Restrictions	4
2.3	Pretty Printing	5
3	Future work	5
4	Acknowledgements	6

1 Syntax

1.1 Lexical Conventions

The source file for the program should be written using Unicode UTF-8 encoding and saved as a file with the `.hs` file extension.

Whitespace

White space characters include space (Unicode 0x20), line feed (0x0A), carriage return (0xD). Four spaces in sequence define an *indent*. Indents are tokens used to define scopes within Haskell.

Comments

The two types of comments are *line comments* and *multi-line comments*. Line comments start on a line when the sequence `--` is detected and continue until the next line feed character. Multi-line comments start when the sequence `{-` is detected and end after `-}` is detected. Nested multi-line comments are not allowed.

Identifier

An *identifier* is a sequence of letters, digits, and underscore characters. The first character must be a letter or underscore. The following regular expression describes valid identifiers: `[_a-zA-Z][_a-zA-Z0-9]*`.

Keywords

A *keyword* is a reserved identifier representing a terminal symbol in the abstract syntax tree. These reserved identifiers cannot be used in other contexts. The following is a list of all keywords:

```
where  system  if   then
else
```

Process Constructor

A *process constructor* is a reserved identifier similar to keywords. The names of these process constructors cannot be used in other contexts. The following regular expression describes valid process constructors: `(actor[1-4][1-4]|delay)SDF`. Only process constructors related to the SDF models defined by ForSyDe Shallow are currently valid.

Integers

An *integer* is a sequence of digits. If the sequence is longer than one digit, the starting digit cannot be a zero. The following regular expression describes valid integers: `0|[1-9][0-9]*`.

Precedence Rules

The order of precedence used to disambiguate the grammar defined in this specification is provided in the table below. The table is ordered such that the top-most row has the highest precedence. Operators that share a row have the same precedence.

Operator	Associativity	Operator Arity
f x	Left	N/A
—	Right	Unary
*	Left	Binary
+ —	Left	Binary

1.2 Concrete Syntax

The syntax for describing a subset of Haskell and ForSyDe is defined using a variant of Extended Backus-Naur form. Nonterminals are written in *italic*, keywords are in double quotes, special characters are in single quotes, and a verbose font is used for tokens which carry a value. The repetition of a set of symbols is enclosed by curly braces {...} and can be repeated zero or more times. An optional set of symbols is enclosed within square brackets [...]. The grammar for the subset of Haskell and ForSyDe explored by this project is as follows:

```

unaryOperator → '-'
binaryOperator → '+' | '-' | '*'
type → "Int" | "Signal" type
        | '(' [ type { ',' type } ] ')'
        | '[' [ type { ',' type } ] ']'
inputs → { expression }
outputs → '(' ( Identifier { ',' Identifier } ) ')'
scope → { Indent statement }
expression → Identifier | Int
        | expression binaryOperator expression
        | unaryOperator expression
        | ProcessConstructor inputs
        | '(' [ expression { ',' expression } ] ')'
        | '[' [ expression { ',' expression } ] ']'
        | "if" expression "then" expression "else" expression
        | Identifier inputs
statement → Identifier inputs '=' expression ["where" scope]
        | outputs '=' expression
        | "system" inputs '=' expression ["where" scope]
        | Identifier '::' type { '-'> type }
global → statement
program → { global }

```

2 Semantics

2.1 Abstract Syntax

The abstract syntax for the subset of Haskell and ForSyDe used in this project is outlined within this section. This abstract syntax defines the general semantics of the subset; it can also be used to create an abstract syntax tree. A list of non-terminal symbols is denoted using an overline. Hat notation is used to denote optional non-terminal symbols.

$$\begin{aligned} unaryOperator &::= - \\ binaryOperator &::= + \mid - \mid * \\ type &::= \text{TypeInt} \mid \text{TypeSignal}(\overline{type}) \\ &\quad \mid \text{TypeTuple}(\overline{type}) \\ &\quad \mid \text{TypeList}(\overline{type}) \\ input &::= \text{Input}(\overline{expression}) \\ output &::= \text{Output}(\overline{identifier}, \overline{identifier}) \\ scope &::= \text{Scope}(\overline{statement}) \\ expression &::= \text{ExpressionIdentifier}(\overline{identifier}) \mid \text{ExpressionInt}(\overline{int}) \\ &\quad \mid \text{ExpressionBinaryOperator}(\overline{binaryOperator}, \overline{expression}, \overline{expression}) \\ &\quad \mid \text{ExpressionUnaryOperator}(\overline{unaryOperator}, \overline{expression}) \\ &\quad \mid \text{ExpressionProcessConstructor}(\overline{processConstructorName}, \overline{input}) \\ &\quad \mid \text{ExpressionTuple}(\overline{expression}) \\ &\quad \mid \text{ExpressionList}(\overline{expression}) \\ &\quad \mid \text{ExpressionIf}(\overline{expression}, \overline{expression}, \overline{expression}) \\ &\quad \mid \text{ExpressionIdentifier}(\overline{identifier}, \overline{input}) \\ statement &::= \text{StatementInput}(\overline{identifier}, \overline{input}, \overline{expression}, \widehat{\overline{scope}}) \\ &\quad \mid \text{StatementOutput}(\overline{output}, \overline{expression}) \\ &\quad \mid \text{StatementSystem}(\overline{input}, \overline{expression}, \widehat{\overline{scope}}) \\ &\quad \mid \text{StatementType}(\overline{identifier}, \overline{type}, \overline{type}) \\ global &::= \text{statement} \\ program &::= \text{Program}(\overline{global}) \end{aligned}$$

2.2 Programmer Restrictions

The following is a set of programmer restrictions which limit what the compiler accepts as input Haskell and ForSyDe code.

- "where" scopes are not allowed to be nested except for the initial module scope.
- "if" expressions are not allowed inside "system"
- Net lists can only be defined using the identifier "system"
- Type signatures are required for all defined functions

2.3 Pretty Printing

This section provides a concrete syntax of the abstract syntax to be able to unambiguously pretty print the AST. This will ensure a standard output that can be used to test the AST for correctness.

```
unaryOperator → "-"
binaryOperator → "+" | "-" | "*"
type → "TypeInt" | "TypeSignal" "(" type ")"
      | "TypeTuple" "(" {type} ")"
      | "TypeList" "(" {type} ")"
input → "Input" "(" {expression} ")"
output → "Output" "(" TextString "," {TextString} ")"
scope → "Scope" "(" {statement} ")"
expression → "ExpressionIdentifier" "(" TextString ")" | "ExpressionInt" "(" Integer ")"
            | "ExpressionBinaryOperator" "(" binaryOperator "," expression "," expression ")"
            | "ExpressionUnaryOperator" "(" unaryOperator "," expression ")"
            | "ExpressionProcessConstructor" "(" TextString "," Input ")"
            | "ExpressionTuple" "(" {expression} ")"
            | "ExpressionList" "(" {expression} ")"
            | "ExpressionIf" "(" expression "," expression "," expression ")"
            | "ExpressionIdentifier" "(" TextString "," input ")"
statement → "StatementInput" "(" TextString "," input "," expression "," [scope] ")"
            | "StatementOutput" "(" Output "," expression ")"
            | "StatementSystem" "(" input "," expression "," [scope] ")"
            | "StatementType" "(" TextString "," type "," {type} ")"
global → statement
program → "Program" "(" {global} ")"
```

3 Future work

The following is a list of potential future features of Haskell and ForSyDe to implement. Use the list to expand the subset of accepted Haskell language features and ForSyDe models. The list is divided into features that should be prioritised and those which are optional.

Prioritized

- Float types
- AbstExt (Prst x, Abst)
- Discard
- Pattern matching

- Synchronous process constructors
- Skeleton-based parallelism

Optional

- Lambda functions
- General types
- Nested "where" scopes

4 Acknowledgements

This reference manual is heavily inspired by the Cigrid Language Reference Manual created by David Broman for the Compilers and Execution Environments (ID2202) course at the Royal Institute of Technology (KTH).